

Distributed Frameworks:

MapReduce

Motivating scenario

Main reasons why MapReduce was introduced in the early 2000's.

- There was an **increasing need to analyze large amounts of data** in a wide spectrum of domains.
- **Big data processing required high storage and compute capabilities** often beyond those of available tools and commonly used platforms
- **Supercomputers, typically used for scientific computing, were unsuited** for a number of reasons:
 - **High costs** (millions/billions euros) to buy and maintain them
 - **Sophisticated programming skills** needed to exploit their potential
 - **Crucial low-level issues** to be addressed by the ~~user~~ **application**: fault tolerance (see next slide), task allocation, data distribution
 - **Fast obsolescence**
 - **Aim for extreme performance**, often not required by typical big data applications

Mean-Time Between Failures (MTBF)

Consider a system of N components $C_1, C_2, \dots, C_N$ that at each time t may fail independently. let

$$p = \text{Prob}(C_i \text{ fails at time } t) \text{ for any } i \text{ and } t$$
$$\Rightarrow \text{Prob}(\exists C_i \text{ that fails at time } t) = 1 - (1-p)^N$$

let $X = \# \text{ time units before the next failure occurs}$

Since the components fail independently, X has a

GEOMETRIC DISTRIBUTION

$$E[X] = \frac{1}{1 - (1-p)^N} \xrightarrow{N \rightarrow +\infty} 1$$

Observation: Big data applications usually run on systems with high storage and computing power, which comprise many individual components. The above simple calculations provide evidence that faults in these systems are likely to occur. Hence, fault tolerance becomes a very relevant issue for big data computing.

MapReduce

- To address the above issues, in 2004 Google introduced **MapReduce** as a **programming framework for big data processing on distributed platforms**
- **Main features:**
 - It can be **employed on moderate cost platforms** (e.g., clusters) on premises or from clouds.
 - Offers a **simple programming environment** inspired by *functional programming* (map, reduce functions)
 - Entrusts **task allocation data distribution, fault-tolerance, load-balancing** issues to the underlying run-time support
 - Adopts a **data-centric view**

MapReduce

The original formulation of MapReduce (see [DG08]) contained

- A **programming model**
- An **implementation** tailored towards a cluster-based computing environment.

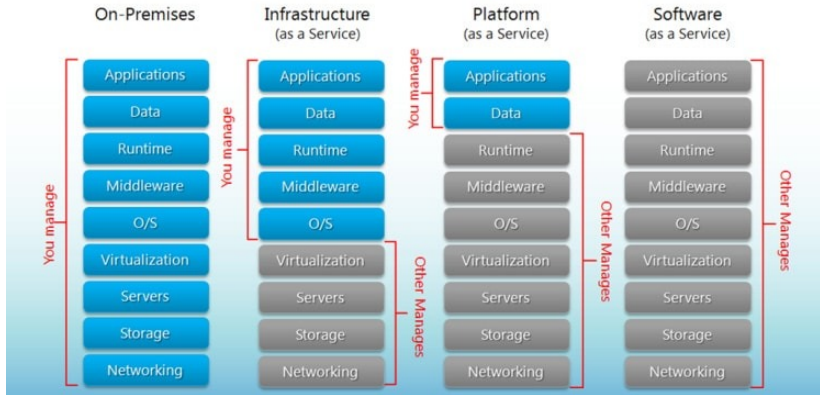
Since its early days, MapReduce evolved into an ecosystem of solutions (e.g., Apache Spark) widely used for big data processing for a **wide array of big-data applications**

Platforms

Typically, MapReduce applications run on

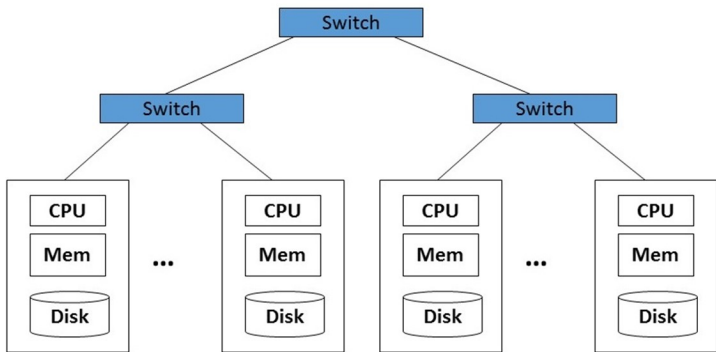
- Clusters of commodity processors (On-premises)
- Platforms from cloud providers (Amazon AWS, Microsoft Azure)
 - IaaS (*Infrastructure as a Service*): provides the users computing infrastructure and physical (or virtual) machines. This is the case of CloudVeneto which provides us a cluster of virtual machines.
 - PaaS (*Platform as a Service*): provides the users computing platforms with OS; execution environments, etc.

Platforms

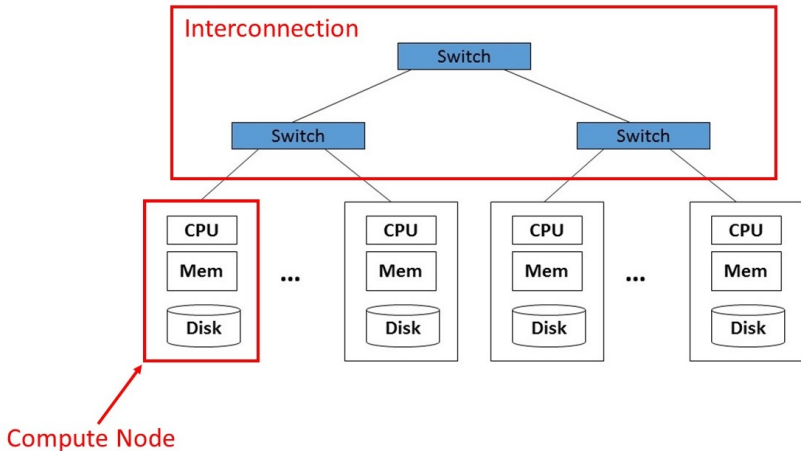


Typical cluster architecture

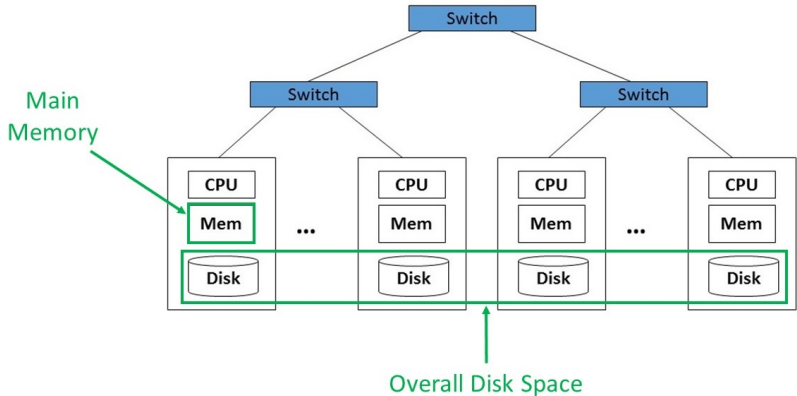
Racks of 16-64 **compute nodes** (commodity hardware), connected by fast switches (e.g., 10 Gbps Ethernet) within/among racks



Typical cluster architecture



Typical cluster architecture



- **Local Main Memories:** used to process small chunks of data at individual compute nodes.
- **Overall Disk Space:** used to store massive inputs (with fault-tolerance)

Distributed File System (DFS)

In a big data applications, file containing the data to be analyzed might be **too big to reside in one compute node**.

Thus, it is convenient, to store the files in a **Distributed File System** across several compute nodes.

- Files divided into *chunks* (e.g., 64MB per chunk).
- Each chunk replicated (e.g., 2x o 3x) with replicas in different nodes to ensure **fault-tolerance**.
- Examples: Google File System (GFS); **Hadoop Distributed File System (HDFS)**

MapReduce-Hadoop-Spark

Several software frameworks have been proposed to support MapReduce programming. For example:

- **Apache Hadoop:**  most popular MapReduce implementation (*often used as synonymous of MapReduce*) from which an entire ecosystem of alternatives has stemmed, aimed at improving its (initially) very poor performance. The Hadoop Distributed File System is still widely used.
- **Apache Spark** (*learned in this course*): one of the most popular and widely used frameworks for big-data applications. It supports the implementation of MapReduce computations, but provides a much richer programming environment. It uses the HDFS.

In this course we will use

- * MapReduce as a CONCEPTUAL MODEL for high-level algorithm design and analysis
- * Spark as a PROGRAMMING MODEL to implement and optimize MapReduce computations

MapReduce computation

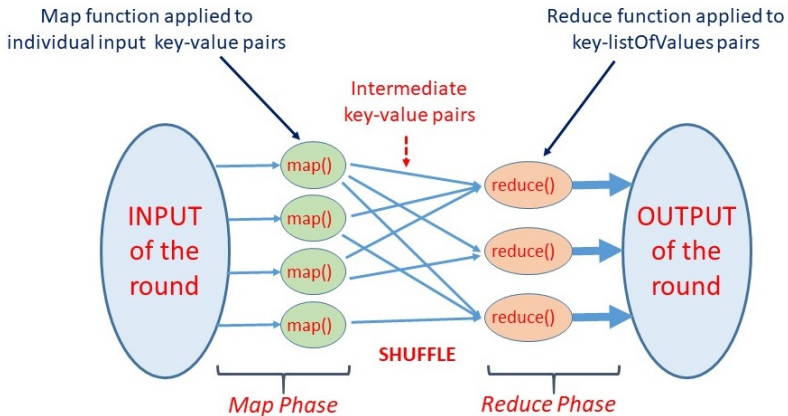
A MapReduce computation can be viewed as a **sequence of rounds**.

A **round** transforms a set of **key-value pairs** into another set of **key-value pairs** (*data-centric view* !), through the following two phases

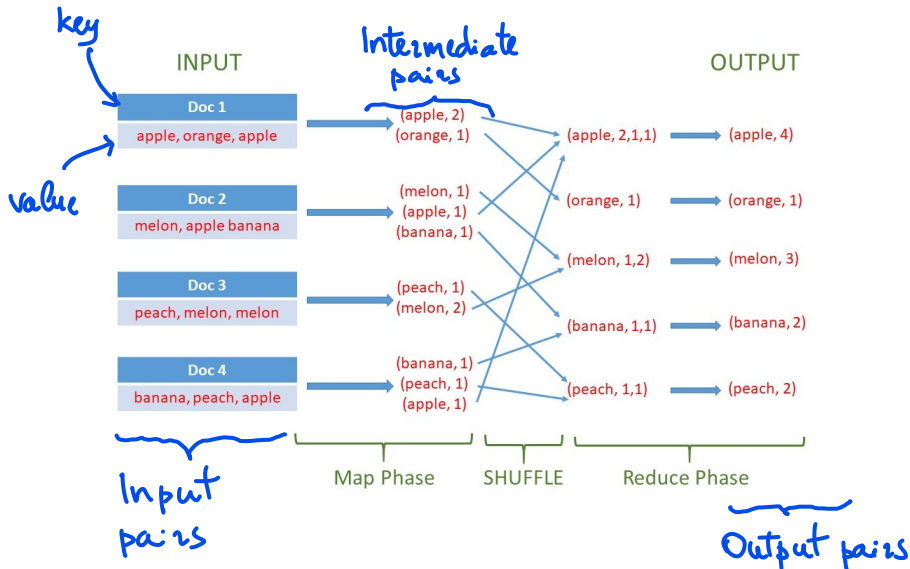
- **Map phase**: for each input key-value pair separately, a user-specified **map function** is applied to the pair and produces ≥ 0 other key-value pairs, sometimes called **intermediate pairs**.
- **SHUFFLE**: intermediate pairs are grouped by key, by creating, for each key k , a **list L_k of values of intermediate pairs with key k** .
- **Reduce phase**: for each key k separately, a user-specified **reduce function** is applied to (k, L_k) which produces ≥ 0 key-value pairs, which is the output of the round.

Terminology: the application of the reduce function to a pair (k, L_k) is referred to as **reducer**.

One round: high-level description

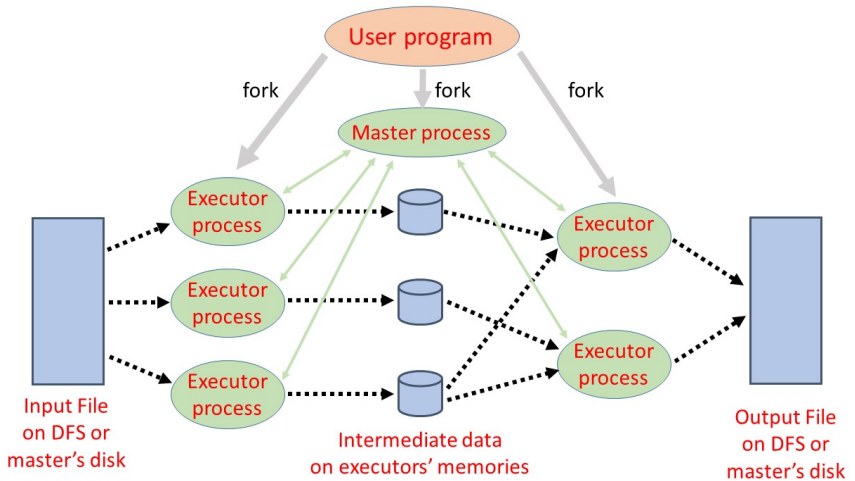


Example: Word count



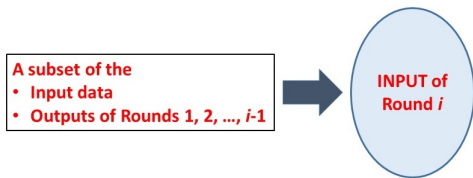
One round: execution on a distributed platform

- The user program is forked into a **master process** and several **executor processes**. The master is in charge of assigning map and reduce tasks to the various executors, and to monitor their status (idle, in-progress, completed).
- **Input files** are read from either the Distributed File System or the master's disk, while intermediate data are stored on the executors' local memories/disks.
- **Output files** (if any) might be stored either on a Distributed File System or on the master's disk.
- **Map phase**: each executor is assigned a subset of input pairs and applies the map function to them sequentially, one at a time.
- **Shuffle**: the system collects the intermediate pairs from the executors' local spaces and groups them by key.
- **Reduce phase**: each executor is assigned a subset of (k, L_k) pairs, and applies the reduce function to them sequentially, one at a time.



Multiple Rounds and Shared Variables

- A MapReduce computation may require **multiple rounds**. If so, the **input of a round** comprises input data and/or data from the outputs of the previous rounds. (Typically, for $i > 1$, the input of Round i is the output of Round $i - 1$.)



- It is often convenient to define **shared variables**, available to all executors, which maintain global information, possibly across different rounds.

Why key-value pairs?

The use of key-value pairs in MapReduce seems an unnecessary complication, and one might be tempted to regard individual data items simply as *objects* belonging to some domain, without imposing a distinction between keys and values.

Justification. MapReduce is **data-centric** and focuses on data transformations which are independent of where data actually reside. The **keys** are needed as **addresses** to reach the objects, and as **labels** to define the groups in the reduce phases.

Practical considerations.

- One should **choose the key-value representations in the most convenient way**. The domains for key and values may change from one round to another.
- Programming frameworks such as **Spark**, while containing explicit provisions for handling key-value pairs, **allow also to manage data without the use of keys**. In this case, internal addresses/labels, not explicitly visible to the programmer, are used.

Specification of a MapReduce algorithm

A *MapReduce (MR) algorithm* should be specified so that

- The **input and output** of the algorithm is **clearly defined**
- The **sequence of rounds** executed for any given input instance is unambiguously implied by the specification
- **For each round** the following aspects are clear
 - **input, intermediate and output** sets of key-value pairs
 - **functions applied in the map and reduce phases.**
- Meaningful values (or asymptotic) bounds for the key performance indicators (defined later) **can be derived.**

Pseudocode style

To specify a MR algorithm with a fixed number of rounds R , we will use the following style:

Input: description of the input as set of key-value pairs

Output: description of the output as set of key-value pairs

Round 1:

- *Map phase*: description of the function applied to each key-value pair
- *Reduce phase*: description of the function applied to each group of key-value pairs with the same key

Round 2, 3, ..., R: similarly

Observation: for simplicity, we sometime provide a high-level description of a round, which, however, must enable a straightforward yet tedious derivation of the Map and Reduce phases.

Example: Word count

Input : k documents $D_1, D_2, \dots, D_k$, where

$D_i \equiv (\text{name}, \text{list of values})$

key 

value 

Output : $\{(w, c(w)) : w \text{ is a word occurring in the } D_i\text{'s}$
 $c(w) = \# \text{ occurrences of } w \text{ in the } D_i\text{'s}\}$

Round 1

Map Phase: for each D_i , $1 \leq i \leq K$, separately

$$D_i \rightarrow \{(w, c_i(w)) : w \in D_i, c_i(w) = \text{\# occurrences of } w \text{ in } D_i\}$$

↑
map function

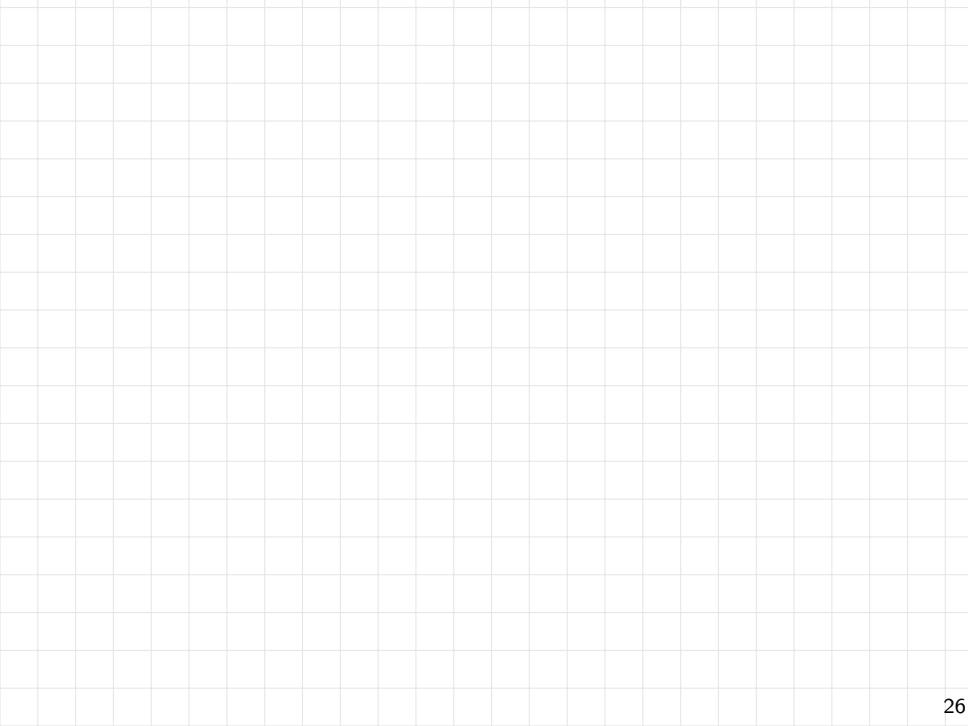
Reduce Phase: for each word w separately, let

$L_w = \text{list of } c_i(w) \text{ } i=1, 2, \dots, K \text{ (some may not exist)}$

$$(w, L_w) \rightarrow (w, c(w) = \sum_{c_i(w) \in L_w} c_i(w))$$

↑
reduce
function

output pair



Performance analysis of a MapReduce algorithm

The analysis of an MR algorithm aims at estimating the following **key performance indicators**:

- **Number of rounds R .**
- **Local space M_L :** *maximum amount of main memory required by a single invocation of a map or reduce function, for storing the input and any data structure needed by the invocation (including possible shared variables). The maximum is taken over all rounds and all invocations in each round.*
- **Aggregate space M_A :** *maximum amount of (disk) space which is occupied by the stored data at the beginning/end of a map or reduce phase. The maximum is taken over all rounds.*

Observations

- * R : rough estimate of RUNNING TIME, under the assumption that all rounds have approximately the same complexity (e.g. dominated by the shuffle)
- * M_L : amount of MAIN MEMORY needed at each executor, hence at each compute node
- * M_A : overall amount of DISK SPACE needed in the system

Analysis of Word Count

$$* R = 1$$

$$* M_L: \quad (\text{Hyp. 1 word} \equiv O(1) \text{ space})$$

$$- N_i = \# \text{ words in } D_i$$

$$- N_{\max} = \max \{ N_i : 1 \leq i \leq K \}$$

$$\text{Map Phase: } M_L = O(N_{\max})$$

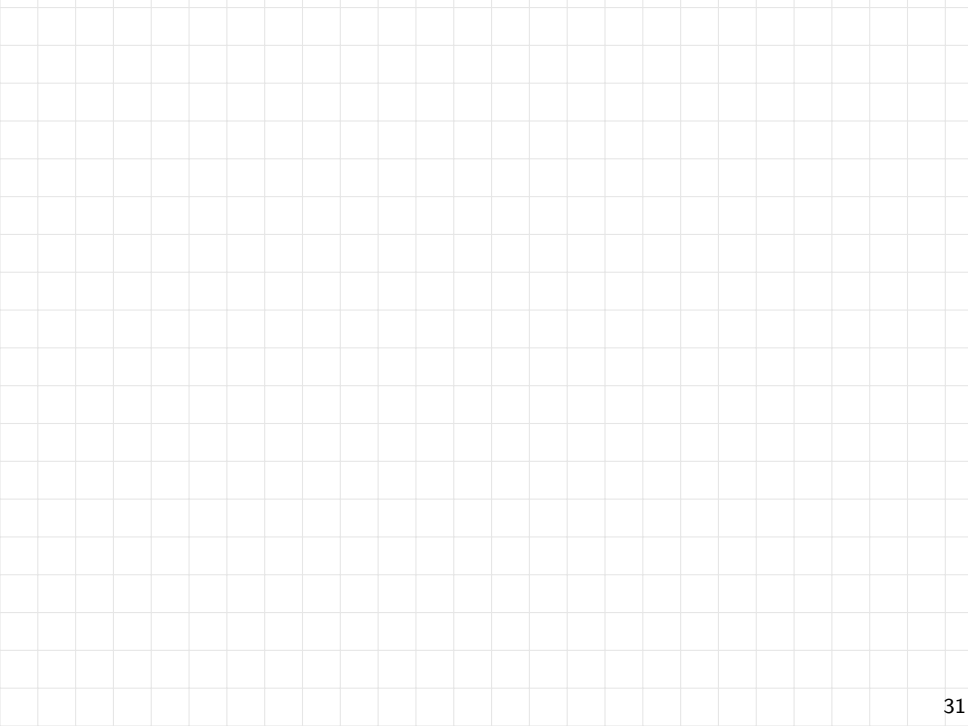
$$\text{Reduce Phase: } M_L = O(K)$$

$$\Rightarrow M_L = O(\max \{ N_{\max}, K \}) = O(N_{\max} + K)$$

* M_A

$N = \sum_{i=1}^k N_i \equiv$ total number of occurrences
of all words aggregated over all documents

- Space required by input pairs: $O(N)$
 - Space required by intermediate pairs: $O(N)$
 - Space required by output pairs: $O(N)$
- $\Rightarrow M_A = O(N)$



Design goals for MapReduce algorithms

Important observation

For every problem solvable by a sequential algorithm in space S there exists a 1-round MapReduce algorithm with both M_L and M_A proportional to S . Justify this observation.

Assign key 0 to all input elements,
and use the sequential algorithm as
reduce function

Remark: the trivial solution implied by the observation is not practical for very large inputs S for the following reasons:

- Very large main memory at each node is needed. Since $S \geq \text{input size}$
- No parallelism is exploited.

Design goals for MapReduce algorithms

Design Goals for MapReduce Algorithms

- 1 Few rounds (e.g., $R = O(1)$);
- 2 Sublinear local space (e.g., $M_L = O(|\text{input}|^\epsilon)$, with $\epsilon < 1$);
- 3 Linear aggregate space (i.e., $M_A = O(|\text{input}|)$), or only slightly superlinear;
- 4 Low complexity of each map or reduce function.

Design goals for MapReduce algorithms

1, 4 : AIM AT TIME EFFICIENCY

1 $\Rightarrow$ low communication/synchronization costs

4 $\Rightarrow$ low computation costs

2, 3 : AIM AT SPACE EFFICIENCY

2 $\Rightarrow$ low main memory usage

3 $\Rightarrow$ low overall space usage
(e.g., low data replication)

} aim at feasibility
of computation on
commodity HW

Observation: the above goals are somewhat conflicting and, in practice, suitable tradeoffs must be sought.

Pros and cons of MapReduce

Why is MapReduce **suitable for big data processing**?

- **Data-centric view.** Algorithm design focuses on data transformations targeting the above design goals.
- **Usability.** The programmer is lifted from performing crucial but difficult activities such as allocation of tasks to workers, data management, and handling of failures, which are instead dealt with transparently by the supporting framework.
- **Portability/Adaptability.** Applications can run on different platforms and the supporting framework will do best effort to exploit parallelism and minimize data movement costs.
- **Cost.** MapReduce applications can be run on moderately expensive platforms, and many popular cloud providers support their execution.

Pros and cons of MapReduce

What are the **main drawbacks** of MapReduce?

- **Running time is only coarsely captured by R .** More sophisticated (yet, less usable) time-performance measures should be used.
- **Curse of the last reducer.** In some cases, one or a few reducers may be much slower than the others, thus delaying the end of the round. Algorithm designers should aim at balancing the load (if at all possible) among reducers.
- MapReduce is **not suitable for applications that require very high performance** and a tight coupling with the underlying architecture (e.g., from computational sciences).

Partitioning

A typical approach to attain the stated goal on local space is to

- Break the computation into small tasks, each processing a small chunk of data (e.g., applications of map/reduce functions)
- Chunks (e.g., for reduce functions) can be created through either deterministic or random partitioning
- Maximize independence among tasks to exploit parallelism

Observation. While partitioning increases parallelism and reduces the local memory requirements, it might increase the number of rounds and, more rarely, the aggregate space.

⇒ Suitable tradeoffs must be sought.

DETERMINISTIC PARTITIONING

Class count

Problem: given a set S of N objects with class labels, count how many objects belong to each class.

More precisely:

Input: Set S of N objects represented by pairs $(i, (\gamma_i, o_i))$, for $0 \leq i < N$, where γ_i is the class of the i -th object (o_i).

Output: The set of pairs $(\gamma, c(\gamma))$ where γ is a class labeling some object of S and $c(\gamma)$ is the number of objects of S labeled with γ .

Remark: we will always assume that N (the number of input pairs) is a known value, i.e., a shared variable.

Class count in 1 round (naive - no partitioning)

Map Phase : $\forall i=0 \dots N-1 \quad (i, (x_i, 0)) \rightarrow (x_i, 1)$

Reduce Phase : For each class Y

$L_Y =$ list of i 's from intermediate pairs $(x_i, 1)$

$(Y, L_Y) \rightarrow (Y, c(Y) = |L_Y|)$

Analysis (assume input pairs take $O(1)$ space)

$R = 1$

M_L : Map Phase: $O(1)$

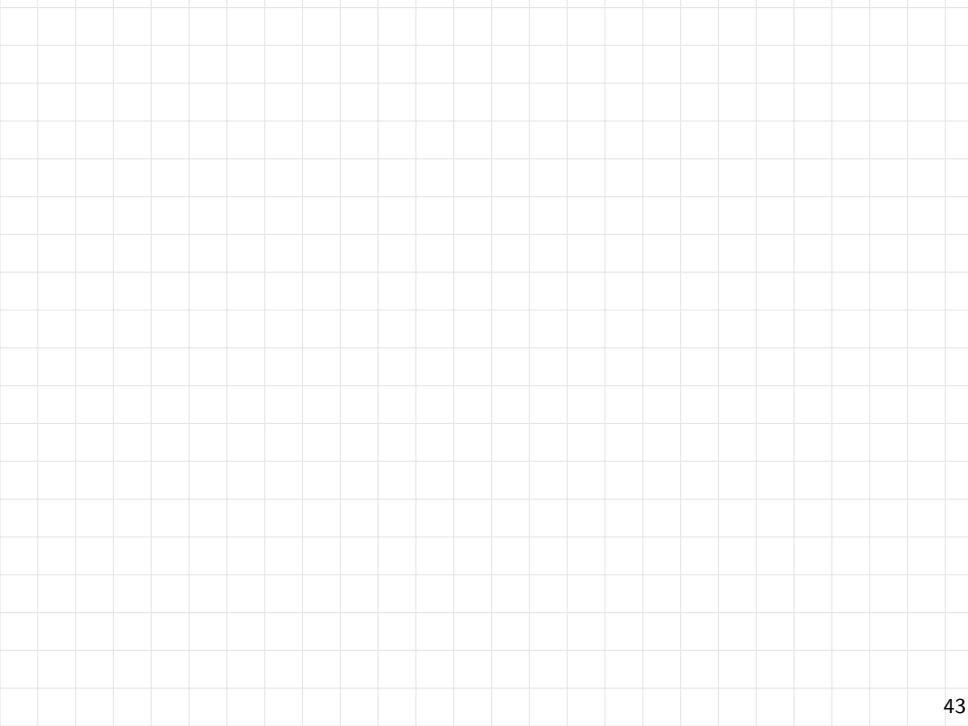
Reduce phase: $O(\text{size of bucket list})$

$\Rightarrow M_L = O(N)$ in the worst case

M_A : $O(N)$ since there are $O(N)$ input, intermediate and output pairs

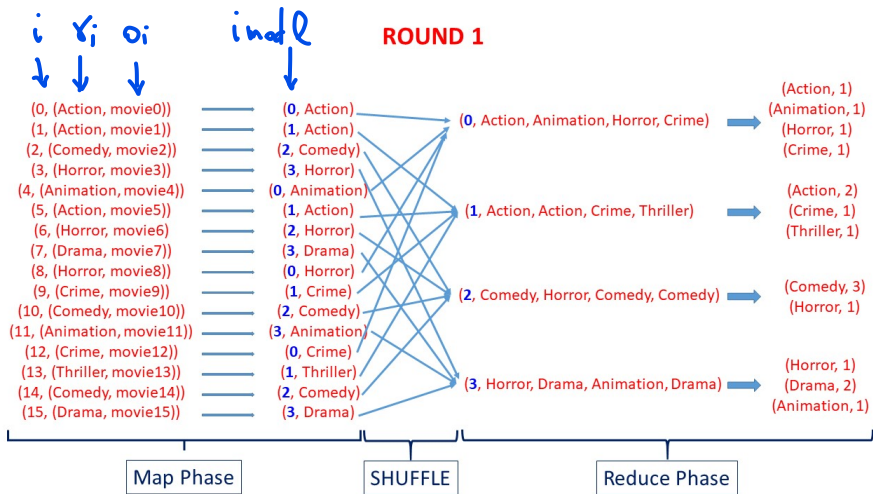
Observation: The algorithm does not comply with the design goal:

$$M_L = O(|\text{input}|^\varepsilon) \quad \varepsilon < 1$$

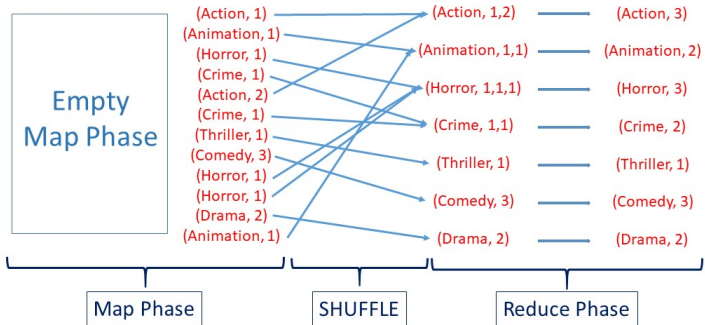


Class count in 2 rounds with deterministic partitions

(example: $\ell = 4$ partitions)



ROUND 2



Class count in 2 rounds with deterministic partitions

Round 1

Map Phase : $\forall 0 \leq i < N$ separately $(i, (r_i, o_i)) \rightarrow (i \bmod l, r_i)$

Reduce Phase : $\forall 0 \leq j < l$ separately, let

L_j : list of class labels r from intermediate pairs
with key j

$(j, L_j) \rightarrow \{ (r, c(j, r)) : r \in L_j, c(j, r) = \text{# occurrences of } r \text{ in } L_j \}$

Round 2

Map Phase: empty

Reduce Phase: x clear x separately, let

$L_x = \text{list of } c(j, x) \text{'s from output pairs}$
 $\text{of Round 1, } 0 \leq j < l \text{ (some may not exist)}$

$$(x, L_x) \rightarrow (x, c(x) = \sum_{c(j, x) \in L_x} c(j, x))$$

Analysis

$$R = 2$$

M_L :

$$\text{Round 1} \left\{ \begin{array}{l} \text{Map Phase} : O(1) \\ \text{Reduce Phase} : O(N/l) \end{array} \right.$$

$$\text{Round 2} \quad \text{Map Phase} : -$$

$$\text{Reduce Phase} : O(l)$$

$$\Rightarrow M_L = O(\max\{N/l, l\}) = O(N/l + l)$$

By fixing $l = \sqrt{N}$ we get $M_L = O(\sqrt{N})$

MA : $O(N)$ since there are $O(N)$ input, intermediate and output pairs in each of the 2 bounds, and pairs take $O(1)$ ops each

$\Rightarrow$ THIS SOLUTION MEETS ALL DESIGN GOALS

Observations

- Deterministic partitioning is attained through a sort of **deterministic hash function**, which uses explicit features, if available, (e.g., the integer key i) to evenly subdivide the pairs into groups.
- The choice of $\ell = \sqrt{N}$ partitions (which we will often make) provides an **example of a simple tradeoff between local space and number of rounds**.
- If more **stringent bounds on the local space** are imposed, different partitionings may be needed, yielding **different round-space tradoffs**. One of the exercises will explore this.
- **In practice, the number of partitions is fixed as a multiple of the number of available workers**, as long as local space is not an issue.

RANDOM PARTITIONING

Consider the class count problem, but assume that the input pairs are provided **without the integer keys in $[0, N)$** .

Namely:

Input: Set S of N objects represented by pairs (γ_i, o_i) , for $0 \leq i < N$, where γ_i is the class of the i -th object (o_i).

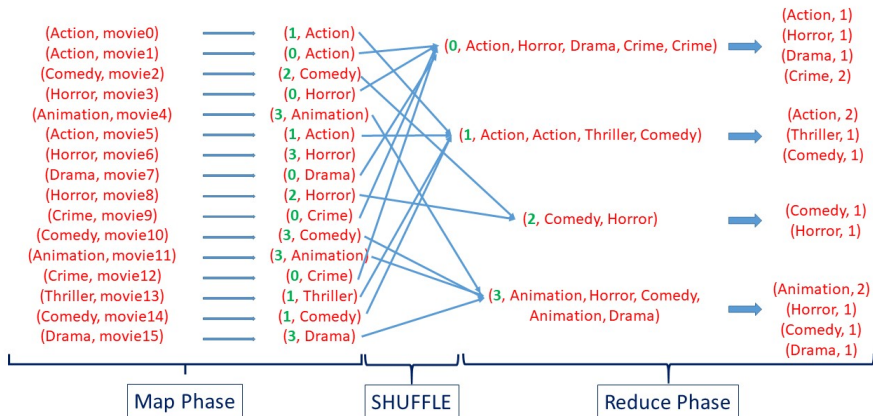
Output: The set of pairs $(\gamma, c(\gamma))$ where γ is a class labeling some object of S and $c(\gamma)$ is the number of objects of S labeled with γ .

We can employ the **2-round approach** seen before, but the partitioning in Round 1 will be achieved now by assigning a **random key in $[0, \ell)$ to each of the intermediate pair**.

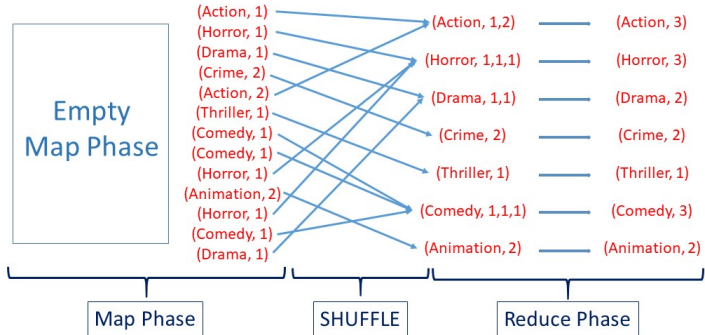
Class count in 2 rounds with random partitions

(example: $\ell = 4$ partitions)

ROUND 1



ROUND 2



Class count in 2 rounds with random partitions

Round 1

Map Phase : $\forall$ input pairs separately $(x_i, o_i) \rightarrow (x, y_i)$

x = random integer in $[0, l)$ with uniform distribution

Reduce Phase : $\forall x \in [0, l)$ separately, let

L_x = list of class labels y from intermediate pairs with key x

$(x, L_x) \rightarrow \{ (y, c(x, y)) : y \in L_x, c(x, y) = \# \text{ occurrences of } y \text{ in } L_x \}$

Round 2 (as before)

Pre-Phase: empty

Reduce Phase: $\forall$ child label γ separately let

$L_\gamma =$ list of $c(x, \gamma)$ from output pairs of Round 1
 $x \in [0, l)$ (some may not exist)

$$(\gamma, L_\gamma) \rightarrow (\gamma, c(\gamma) = \sum_{c(x, \gamma) \in L_\gamma} c(x, \gamma))$$

Analysis

m_x = number of intermediate pairs in Round 1
with key x

$m = \max \{ m_x : 0 \leq x < l \} \rightarrow$ size of largest
partition in R_1

$R = 2$

$M_L:$

Round 1 { Map Phase : $O(1)$
Reduce Phase : $O(m)$

Round 2 { Map Phase : —
Reduce Phase : $O(l)$

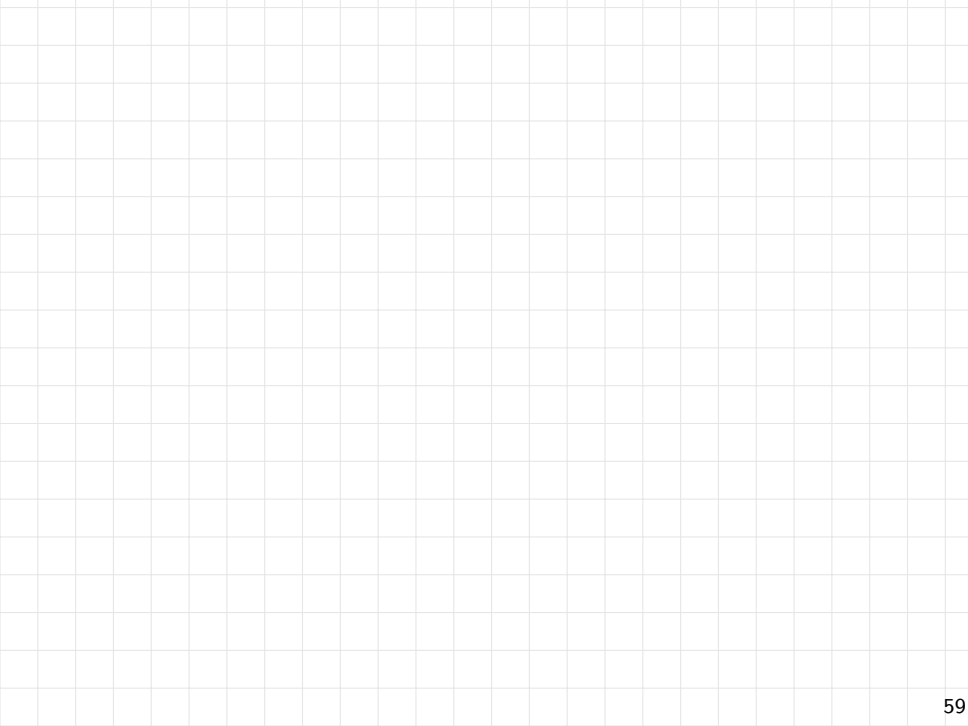
$$\Rightarrow M_L = O(m + l)$$

$M_A : O(N)$ same argument as before

Deterministic partitioning : $M_L = O(N/l + l)$

Random partitioning : $M_L = O(m + l)$

HOW LARGE CAN m be ??



How large can m be?

Theorem

Fix $\ell = \sqrt{N}$ and suppose that in Round 1 the keys assigned to intermediate pairs independently and with uniform probability from $[0, \sqrt{N})$. Then, with probability at least $1 - 1/N^5$

$$m = O\left(\frac{N}{\ell}\right) = O(\sqrt{N}).$$

Based on the theorem (which we will prove shortly) and on the previous analysis, by setting $\ell = \sqrt{N}$ we get

$$M_L = O(m + \ell)$$

$$M_L = O\left(\frac{N}{\ell} + \ell\right) = O(\sqrt{N})$$

with probability at least $1 - 1/N^5$ (*very close to 1*, for large N).

N.B.: This is an example of **probabilistic analysis**, as opposed to more traditional worst-case analysis.

Review of useful probabilistic tools

Union bound

Given a countable set of events $E_1, E_2, \dots, E_r$, we have:

$$\Pr\left(\bigcup_{i=1}^r E_i\right) \leq \sum_{i=1}^r \Pr(E_i).$$

Markov's inequality

Let X be a nonnegative random variable. For any $a > 0$ we have:

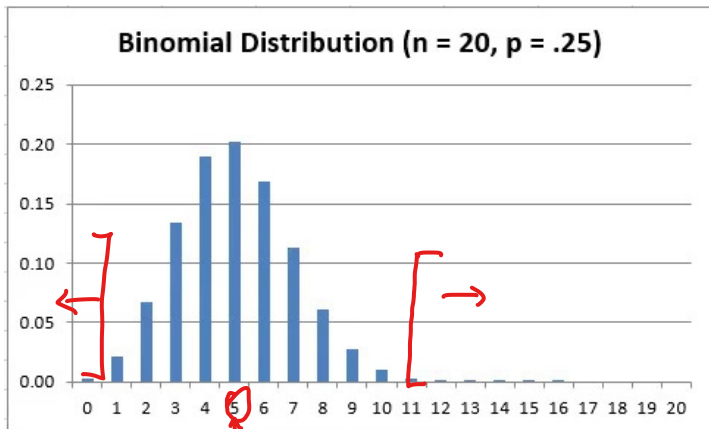
$$\Pr(X \geq a) \leq \frac{\mathbb{E}[X]}{a}$$

Useful probabilistic tools

Chernoff bound

Let $X_1, X_2, \dots, X_n$ be n i.i.d. Bernoulli random variables, with $\Pr(X_i = 1) = p$, for each $1 \leq i \leq n$. Thus, $X = \sum_{i=1}^n X_i$ is a $\text{Binomial}(n, p)$ random variable. Let $\mu = \mathbb{E}[X] = n \cdot p$. For every $\delta_1 \geq 0$ and $\delta_2 \in (0, 1)$ we have:

- (a) $\Pr(X \geq (1 + \delta_1)\mu) \leq 2^{-\delta_1^2 \mu / 3}$
- (b) $\Pr(X \leq (1 - \delta_2)\mu) \leq 2^{-\delta_2^2 \mu / 2}$



Expectation

Proof of theorem

Assume $N \geq 16$ (otherwise theorem is straightforward)

Focus on an arbitrary $x \in [0, \sqrt{N})$ and recall that

$m_x = \#$ intermediate pairs that in R1 receive key x

For $0 \leq i < N$ define $Y_i = \begin{cases} 1 & \text{if } (x_i, o_i) \rightarrow (x, Y_i) \\ 0 & \text{else} \end{cases}$

Y_i is a Bernoulli random variable with $P(Y_i = 1) = \frac{1}{\sqrt{N}}$

$\Rightarrow m_x = \sum_{i=0}^{N-1} Y_i \Rightarrow m_x$ is Binomial $(N, \frac{1}{\sqrt{N}})$

$\Rightarrow E[m_x] = \sqrt{N}$

By the Chernoff Bound (a) we have that

$$P_2(m_x \geq 6\sqrt{N}) \leq 2^{-6\sqrt{N}}$$

Now, for $N \geq 16$ we have $\sqrt{N} \geq \log_2 N$

$$\Rightarrow 2^{-6\sqrt{N}} \leq 2^{-6\log_2 N} = ((1/2)^{\log_2 N})^6 = (1/N)^6$$

$$\Rightarrow P_2(m_x \geq 6\sqrt{N}) \leq (1/N)^6$$

We now extend the bound to $m = \max\{m_x : x \in [0, \sqrt{N}]\}$

Define the event $E_x = "m_x \geq 6\sqrt{N}"$

$$P_2(m \geq 6\sqrt{N}) = P_2(E_0 \cup E_1 \cup \dots \cup E_{\sqrt{N}-1})$$

$$\begin{aligned}
 (\text{by the union bound}) &\leq \sqrt{N} \cdot P_2(E_x) \text{ for any } x \\
 &\leq \sqrt{N} \cdot \frac{1}{N^6} \leq \frac{1}{N^5}
 \end{aligned}$$

Hence the probability $P_2(m \geq 6\sqrt{N}) \leq 1/N^5$

$$\Rightarrow P_2(m < 6\sqrt{N}) \geq 1 - 1/N^5$$

$\Rightarrow$ With VERY HIGH PROBABILITY $m < 6\sqrt{N}$
 that is $m = O(\sqrt{N})$ □

CONCLUSION: If we distribute N objects at random among l partitions, and N/l is sufficiently large, then with high probability no partition receives an excessive number of objects

Observations on partitioning

- **Deterministic partitioning:** applicable whenever input pairs contain pieces of data which can be mapped into partition indices that ensure a sufficiently even partitioning of the pairs.
- **Random partitioning:** always applicable, but to ensure a sufficiently even partitioning, the expected size of each partition should not be too small, and a good random generator must be used.

Theory question from Written Exam (22/23)

A MapReduce algorithm receives in input N web documents, each represented by a pair (url, D) , where D is the document and url is its address. Assume that each pair occupies $O(1)$ space. Describe a map phase that splits the input into $\sqrt{N}$ small partitions, and state the theoretical guarantees of your partitioning.

Exercise (See ex. 2.3.1.(b) of [LRU14])

Let S be a set of N integers represented by pairs (i, x_i) , with $0 \leq i < N$, where i is the key of the pair and x_i is an arbitrary integer.

- 1 Design a 2-round MR algorithm to compute the arithmetic mean of the integers in S , using $O(\sqrt{N})$ local space and $O(N)$ aggregate space.
- 2 Show how to modify the algorithm of the previous point to reduce the local space bound to $O(N^{1/4})$, at the expense of an increased number of rounds.
- 3 Can you attain $M_L = O(M)$, for any $M < \sqrt{N}$?

Exercise (See ex. 2.3.1.(d) of [LRU14])

Let S be a set of N integers represented by pairs (i, x_i) , with $0 \leq i < N$, where i is the key of the pair and x_i is an arbitrary integer. We want to design an efficient MR algorithm to compute the number D of distinct integers in S .

- 1 Design a simple 2-round MR algorithm to compute D and analyze its local and aggregate space requirements. Under what circumstances is the local space proportional to N , thus not meeting the design goals of MR algorithms?
- 2 Design a better MR algorithm to compute D in $O(1)$ rounds, using $O(\sqrt{N})$ local space and $O(N)$ aggregate space.

Exercise

Let P be a set of N temperatures measured by several sensors. Each element of P is a pair $(i, (s_i, temp_i))$, where $i \in [0, N - 1]$ is a distinct integer key, s_i is a sensor, and $temp_i$ is a temperature. No assumptions are made on the number of pairs in P from the same sensor.

- 1 Let $K > 0$ be an integer constant. Design a 2-round MapReduce algorithm that, given P , returns a subset T of values $(s_i, temp_i)$ from pairs in P , selected as follows. For every sensor s , if in P there are $\leq K$ pairs $(i, (s_i, temp_i))$ with $s_i = s$, then T must contain the values of all these pairs, otherwise T must contain the values of exactly K of them, arbitrarily selected.
- 2 Analyze the local and aggregate space required by your algorithm. For full score, your algorithm must require $o(N)$ local space and linear aggregate space.
- 3 How does the local and aggregate space of your algorithm change if $K = \log_2 N$?

Exercise

Design an $O(1)$ -round MR algorithm for computing a matrix-vector product $W = A \cdot V$, where A is an $m \times n$ matrix, V is an n -vector, and $m \leq \sqrt{n}$. Your algorithm must use $o(n)$ local space and linear (i.e., $O(mn)$) aggregate space.

How would your solution change if m were larger?

Summary

- MapReduce.
 - Motivating scenario.
 - Main Features, platforms and software frameworks.
 - MapReduce computation: high-level structure, algorithm specification, analysis, key performance indicators.
 - Algorithm design goals.
- Partitioning:
 - Deterministic partitioning
 - Randomized partitioning
 - Probabilistic tools: union bound, Markov's inequality, Chernoff bounds.